

PHARMACY MANAGEMENT SYSTEM

Complete Project Overview · OOP Lab Project · CSP-122

12

Java Files

4

OOP Concepts

5

Data Files

2

User Roles

19

Preloaded
Medicines

8

Key Features

1. What Is This Project?

Purpose, scope and problem it solves

The Pharmacy Management System is a fully functional console-based Java application built as part of the Object Oriented Programming course (CSP-122) at the Institute of Business Management. It is designed to replace the manual, paper-based operations of a small pharmacy with a single organized program that manages medicines, sales, billing, stock alerts, user access, and file persistence.

Most small pharmacies in Pakistan still track stock in notebooks, miss expiry dates, have no separation between what an owner can do versus what a staff member can do, and generate no proper records. This project addresses all of those problems in one clean, well-structured Java application.

This is a desktop application — not a website. It runs through the terminal on any computer with Java installed. When the GUI module is taught, only the Menu class needs to be replaced with Swing windows — every other class stays exactly as it is.

2. System Features

Everything the system can do

Inventory Management

- View all medicines in a formatted table — ID, name, company, price, expiry, quantity, formula, prescription status
- Add new medicines with all details including drug formula category and prescription flag
- Update existing medicine price and quantity
- Delete medicines — restricted to Admin only with confirmation prompt
- Inline low stock warning displayed while viewing the inventory table

Search and Alternative Suggestions

- Search by partial name keyword — typing 'pan' finds Panadol
- Search by exact medicine ID for quick lookup
- If a medicine is out of stock, the system scans for alternatives with the same drug formula category
- Alternatives displayed with name, quantity, and price — no dead ends for the customer

Sales and Billing

- Sell any medicine by entering its ID and quantity
- Bill generated automatically — bill ID, medicine, quantity, price per unit, total, date and time, sold by
- Full formatted receipt printed on screen after every transaction
- Apply percentage discount to any bill — Admin only, applied after the sale
- Refund processing — reverses the sale, restores stock to inventory, marks bill as refunded

Sales Reports — Admin Only

- Daily summary — total transactions and revenue for today
- Weekly summary — last 7 days transactions and revenue
- Monthly summary — current month transactions and revenue
- Best selling medicine — which medicine sold the most units overall
- Profit and Loss statement — total revenue, total discounts given, total refunded, net earnings

Alert System

- Runs automatically on every program startup before the menu appears
- Flags all expired medicines with the exact expiry date
- Flags medicines expiring within 7 days — shows exact days remaining
- Flags medicines that have fallen below their minimum stock threshold
- All alerts saved to a log file with timestamp for historical reference
- Admin can re-run alert checks manually and view the current alert report anytime

Access Control and Security

- Two user roles — Admin with full access and Staff with restricted access
- Login required with username and password for both roles
- Maximum 3 login attempts before session locks — remaining attempts shown on screen
- Admin only — delete medicines, change prices, process refunds, apply discounts, view reports
- Staff can view, add, update, search, sell, and view alerts

File Persistence

- Inventory saved to file after every change — nothing is ever lost between runs
- Sales log appended after every transaction — never overwritten
- Activity log records every admin action with timestamp
- Alert history saved on every startup
- Dated backup of full inventory created automatically on every program shutdown

Activity Logging and Accountability

- Every admin action is recorded — price changes, deletions, discounts, refunds
- Each entry has a timestamp showing exact date and time
- Admin can view the full activity log from the menu at any time
- Creates a clear trail of who did what and when

3. Project Architecture

How all classes connect

ENTRY POINT: Menu.java – main() starts here

```
|
+■■ LOGIN
|   User.java (abstract base class)
|       +■■ Admin.java – full access
|       +■■ Staff.java – restricted access
|
+■■ STARTUP SEQUENCE
|   FileManager.initializeFolders()
|   FileManager.loadInventory() → fills Medicine array from file
|   DataLoader.loadDefaultMedicines() → first run only
|   AlertSystem.checkAllOnStartup()
|       +■■ DateHelper → all date parsing and comparison
```

```

|
+■■ INVENTORY
|   Medicine.java
|       +■■ Static array of 100 Medicine objects
|       +■■ Add, Delete, Update, Search by name/ID
|       +■■ findAlternatives(formulaCategory)
|
+■■ SALES
|   Sales.java
|       +■■ sellMedicine() → creates Bill, reduces stock
|       +■■ applyDiscount() → admin only, modifies bill
|       +■■ processRefund() → restores stock, marks refunded
|       +■■ Reports: daily, weekly, monthly, best seller, P&L;
|       +■■ Bill.java → one object per transaction
|
+■■ FILE OPERATIONS
|   FileManager.java
|       +■■ inventory.txt
|       +■■ sales_log.txt
|       +■■ activity_log.txt
|       +■■ alerts_log.txt
|       +■■ backups/inventory_DD-MM-YYYY.txt
|
+■■ SHUTDOWN SEQUENCE
|       FileManager.saveInventory()
|       FileManager.generateBackup()

```

4. Class-by-Class Detail

What each class does, its fields, methods, and connections

Medicine

Medicine.java

The foundation of the entire system. Stores all medicine data in a static array of 100 objects. Handles all inventory operations and the alternative medicine suggestion feature.

FIELDS & METHODS

- int id (auto-generated)
- String name, company
- double price
- String expiryDate (dd/MM/yyyy)
- int quantity, minStockThreshold
- String formulaCategory
- boolean prescriptionRequired
- addMedicine() — add to array
- deleteMedicine(id) — shift array
- getMedicineById(id) — for updates
- searchByName() — partial match
- searchById() — exact match
- findAlternatives() — same formula
- viewAll() — formatted table
- reduceStock() / increaseStock()
- toCsvString() / fromCsvString()

LINKED TO / OOP USED

- FileManager — save on every change
- AlertSystem — provides stock data
- Sales — stock reduced on sale
- DataLoader — loads default medicines
- Encapsulation — all fields private
- Array of Objects — inventory array
- Static fields — shared across program

FileManager

FileManager.java

The only class in the system that reads or writes files. All other classes call FileManager for any file operation. Manages five separate data files and the backup system.

FIELDS & METHODS

- INVENTORY_FILE path
- SALES_LOG_FILE path
- ACTIVITY_LOG_FILE path
- ALERT_LOG_FILE path
- BACKUP_FOLDER path
- initializeFolders() — create data/
- loadInventory() — on startup
- saveInventory() — after every change
- logSale() — append to sales log
- logActivity() — append admin action
- logAlert() — append alert
- generateBackup() — dated copy
- printSalesLog() / printActivityLog()

LINKED TO / OOP USED

- Medicine — load/save inventory
- Sales + Bill — log every transaction
- AlertSystem — save alert history
- Admin — save activity log
- Single Responsibility — file ops only
- Exception Handling — all try-catch
- File I/O — core purpose of class

DataLoader

DataLoader.java

Loads 19 hardcoded medicines on first run only. Split into three categories — daily use, high risk prescription, and chronic care. Once saved to file it never runs again.

FIELDS & METHODS

- Static method only — no fields
- 19 medicines across 3 categories
- IDs 1001-1006, 2001-2006, 3001-3007
- loadDefaultMedicines() — one-time load

LINKED TO / OOP USED

- Medicine — calls addMedicine()
- FileManager — triggers saveInventory()
- Called by Menu startup() if count == 0
- Static utility class
- Separation of concerns — data separate from logic

DateHelper

DateHelper.java

Utility class that handles all date-related operations. Uses dd/MM/yyyy format matching Medicine.java. Called exclusively by AlertSystem for all expiry calculations.

FIELDS & METHODS

- DateTimeFormatter FORMATTER = dd/MM/yyyy
- daysUntilExpiry() — returns long
- isExpired() — true if today or past
- isExpiringSoon() — within warning window
- formatForDisplay() — dd MMM yyyy
- todayAsString() — today formatted

LINKED TO / OOP USED

- AlertSystem — sole caller of this class
- Single Responsibility — dates only
- Exception Handling — bad date formats

AlertSystem

AlertSystem.java

Runs automatically on every startup before the menu appears. Checks all medicines for expiry and low stock issues. Saves all alerts to file and displays them to the user.

FIELDS & METHODS

- int expiryWarningDays (default 7)
- String[] alertHistory (max 100)
- int alertCount
- checkAllOnStartup() — entry point
- checkExpiry() — scan all medicines
- checkLowStock() — below threshold
- showAlerts() — print to console
- saveAlertHistory() — via FileManager
- getAlertHistory() / getAlertCount()

LINKED TO / OOP USED

- Medicine — reads inventory array
- FileManager — saves alert log
- DateHelper — all date comparisons
- Menu — called in startup()
- Encapsulation — threshold configurable
- Separation of Concerns — alerts isolated

Represents one complete sale transaction. Calculates totals automatically, prints receipts, and converts to/from CSV for file storage. Tracks refund status.

FIELDS & METHODS

- int billID (auto-incremented)
- int medicineID
- String medicineName
- int quantitySold
- double pricePerUnit
- double discountPercent
- double totalAmount
- String dateOfSale, soldBy
- boolean refunded
- calculateTotal() — auto on create
- printBill() — formatted receipt
- toCsvString() — for file saving
- fromCsvString() — rebuild from file
- setDiscountPercent() — recalculates total
- setRefunded()

LINKED TO / OOP USED

- Sales — created per transaction
- FileManager — saved to sales log
- Encapsulation — all fields private
- Composition — Sales HAS-A Bill
- Single Responsibility — billing only

Manages all selling, discounting, refunding, and reporting. Keeps an in-memory list of all bills loaded from the sales log file. Connects Medicine, Bill, and FileManager.

FIELDS & METHODS

- ArrayList salesLog (Bill objects)
- double totalRevenue
- double totalDiscount
- double totalRefunded
- sellMedicine() — core sell flow
- applyDiscount() — by bill ID
- processRefund() — restore stock
- getDailySummary() / getWeeklySummary()
- getMonthlySummary()
- getBestSellingMedicine()
- calculateProfitLoss()
- viewSalesLog()
- loadSalesHistory() — on construct

LINKED TO / OOP USED

- Medicine — reduce/restore stock
- Bill — created per sale
- FileManager — log sale + save inventory
- Menu — called from handleInput()
- Encapsulation — revenue fields private
- Composition — contains Bill objects
- ArrayList — dynamic sales log

User + Admin + Staff

User.java / Admin.java / Staff.java

User is the abstract base class. Admin and Staff extend it. Controls login, access rights, and capabilities. Login lockout enforced at 3 failed attempts.

FIELDS & METHODS

- String username, password (private)
- String role (Admin or Staff)
- int loginAttempts (max 3)
- abstract showCapabilities()
- login() — validates + tracks attempts
- isLocked() — check lockout status
- Admin.changePrice() — updates + logs
- Admin.viewActivityLog()
- Staff / Admin showCapabilities()

LINKED TO / OOP USED

- Menu — currentUser is User type
- Medicine — Admin controls price/delete
- FileManager — Admin logs activity
- Abstraction — abstract User class
- Inheritance — Admin and Staff extend User
- Polymorphism — User ref at runtime
- Encapsulation — password private

Menu

Menu.java

The entry point and controller of the entire system. Handles all Scanner input and screen output. Connects every class together. Easy to replace with GUI when needed.

FIELDS & METHODS

- Scanner scanner
- Medicine[] inventory
- User currentUser (null until login)
- AlertSystem alertSystem
- Sales sales
- main() — entry point
- showLoginScreen() — 3 attempt login
- startup() — load + alerts + data
- shutdown() — save + backup
- showAdminMenu() / showStaffMenu()
- handleInput() — routes all choices
- addMedicineFlow() / searchMedicineFlow()
- updateMedicineFlow() / deleteMedicineFlow()
- readIntInput() / readDoubleInput()

LINKED TO / OOP USED

- ALL classes — Menu is the controller
- AlertSystem — run on startup
- FileManager — startup load + shutdown save
- Polymorphism — currentUser is Admin or Staff
- Composition — contains all instances
- Separation of UI from logic

5. Class Descriptions

What each class does in plain language

Medicine.java

This is the foundation of everything. It holds all medicine information — name, company, price, expiry date, quantity, minimum stock threshold, formula category, and prescription flag. It stores all medicines in a static array of 100 objects. Static means there is one shared array across the entire program. It provides methods to add, delete, search by name or ID, view all medicines, and find alternatives. The find alternatives method is the unique feature — when a medicine is out of stock it scans the array for other medicines with the same formula category and returns them as suggestions. Two constructors exist — one that auto-generates an ID for new medicines, and one that takes a fixed ID when loading from file so IDs are never duplicated.

FileManager.java

This is the only class that touches files. Every other class calls FileManager when it needs to read or write anything. It manages five things: inventory.txt which is the full list of medicines overwritten on every change, sales_log.txt where every sale is appended with a timestamp and never overwritten, activity_log.txt where every admin action with timestamp is appended, alerts_log.txt where every startup alert is saved, and the data/backups/ folder where a dated copy of inventory is created on every shutdown. All file operations are wrapped in try-catch so the program never crashes if a file is missing or corrupted.

DataLoader.java

This class exists purely to load 19 real medicines on first run. It is split into three categories matching the Excel files prepared by the documentation team member — daily use medicines like Panadol and Brufen, high risk prescription medicines like Morphine and Xanax, and chronic care medicines like Lantus and Glucophage. It only runs once. After that everything persists through FileManager. Two medicines — Lantus and Morphine — have expired or near-expiry dates intentionally. They will trigger the AlertSystem the moment the program starts.

DateHelper.java

A small utility class that handles everything date-related. It parses dates in dd/MM/yyyy format, checks if a medicine is expired, checks if it is expiring within a warning window, formats dates for display, and returns today's date as a string. AlertSystem calls this class for all its date logic. Having date operations in one place means if the date format ever changes, only this one class needs updating.

AlertSystem.java

Runs automatically every time the program starts, before the menu appears. It checks two things — which medicines have expired or are expiring within 7 days, and which medicines have fallen below their minimum stock threshold. If any alerts are found they are printed to screen and saved to the alert log file through FileManager. The warning window is configurable — currently set to 7 days in Menu.java. This ensures critical information is never buried inside a menu option that someone might forget to check.

Bill.java

Represents one complete sale transaction. It stores the bill ID, medicine ID, medicine name, quantity sold, price per unit, discount percentage, total amount, date and time of sale, who sold it, and whether it was refunded. It calculates the total automatically when created and recalculates it if a discount is applied later. It can convert itself to a CSV string for saving and rebuild itself from a CSV string when loading from file. Every sale in the system is represented by exactly one Bill object.

Sales.java

Manages all selling operations. When a sale happens it finds the medicine, checks stock, creates a Bill object, reduces the medicine quantity, saves the updated inventory, logs the sale to file, and prints the receipt. It also handles discounts — applied to a specific bill after the sale — and refunds — which reverse the sale, restore the stock, and mark the bill as refunded. It keeps an in-memory list of all bills loaded from the sales log file. It also has daily, weekly, and monthly summary methods plus a best selling medicine tracker and profit and loss statement.

SalesReport.java

A supporting class that assists Sales with reporting output. Its showProfitLoss() method simply calls sales.calculateProfitLoss() directly. It provides a clean separation between report formatting and the core sales logic, keeping each class focused on a single responsibility.

User.java

Abstract base class for all users in the system. Holds username, password, role, and login attempt count. Has a login() method that validates credentials and tracks failed attempts up to a maximum of 3. Has an abstract method showCapabilities() that every subclass must implement. You cannot create a User object directly — you must create either an Admin or Staff. This is Abstraction in practice.

Admin.java

Extends User with role set to Admin. Gets full access to everything in the system. Has a changePrice() method that updates a medicine's price, saves inventory, and logs the action to the activity file. Has a viewActivityLog() method that prints the full activity log. The showCapabilities() method lists everything an admin can do. Admin is the only role that can delete medicines, change prices, process refunds, apply discounts, and view all reports.

Staff.java

Extends User with role set to Staff. Gets restricted access — can sell, search, view, and add medicines but cannot delete or change prices. showCapabilities() lists the staff permissions clearly. This separation of roles reflects how a real pharmacy operates — the pharmacist handles day-to-day sales while the owner controls pricing and records.

Menu.java

The controller of the entire system. It connects every class together. It handles all Scanner input and all screen output. It shows the login screen, routes to admin or staff menu based on who is logged in, handles every menu choice in `handleInput()`, and calls the correct class method for each action. On startup it loads data and runs alerts. On shutdown it saves inventory and creates a backup. When the GUI module is taught, only this class needs to be replaced with Swing windows — every other class stays exactly as it is because the business logic was kept completely separate from the user interface.

6. OOP Concepts Used and Where

Mapping every concept to exactly where it appears in the code

Concept	Where Used	How It Is Applied
Encapsulation	All 12 classes	Every field across every class is declared private. Access is controlled through methods.
Inheritance	User → Admin, Staff	Admin and Staff extend User. They inherit username, password, role, and methods.
Abstraction	User.java	User is declared abstract. You cannot create a new <code>User()</code> object directly and must use Admin or Staff.
Polymorphism	Menu.java — <code>currentUser</code>	<code>currentUser</code> is declared as type User. At runtime it holds either an Admin or Staff object.
File I/O	FileManager.java	All five data files managed through one class. <code>BufferedReader</code> and <code>BufferedWriter</code> for reading and writing.
Exception Handling	FileManager, Sales, Menu	Every file operation wrapped in try-catch for <code>IOException</code> and <code>FileNotFoundException</code> .
Array of Objects	Medicine.java	A fixed static array of 100 Medicine objects stores the full inventory. This is updated when inventory changes.
ArrayList	Sales.java	An <code>ArrayList</code> of Bill objects stores the in-memory sales log. Loaded from file on startup.
Composition	Sales HAS-A Bill	Sales contains Bill objects — it creates them, stores them, and manages them.
Static Members	Medicine.java	The inventory array, count, and <code>idCounter</code> are static. They belong to the class, not an instance.

7. Data Files

How data is stored and when each file is updated

File	Format	Contents	Updated When
inventory.txt	CSV per line	All medicine records — full inventory state	Add, update, delete, or sell
sales_log.txt	Timestamp CSV	Every bill transaction — never overwritten	Every sale or refund
activity_log.txt	Timestamp text	Every admin action recorded	Every admin action

alerts_log.txt	Timestamp text	All expiry and low stock alerts	Every program startup
backups/inventory_DD-MM-YYYY.txt	CSV per line	Full dated inventory snapshot	Every program shutdown

8. Team Ownership

Which member built which file

File(s)	Owner	Responsibility
Medicine.java, FileManager.java, DataLoader.java	Member 1 — Team Leader	Core inventory engine, all file operations, preloaded
Bill.java, Sales.java, SalesReport.java	Member 2 — Sales	Billing, transactions, discounts, refunds, all reports
User.java, Admin.java, Staff.java	Member 3 — Access Control	Abstract user hierarchy, login, role-based access
AlertSystem.java, DateHelper.java, Menu.java	Member 4 — Integration	Alerts, date utilities, main menu, system controller
Proposal, Report, UML, Presentation	Member 5 — Documentation	All written documentation and presentation material

Pharmacy Management System · OOP Lab Project · CSP-122 · IoBM · Spring 2026